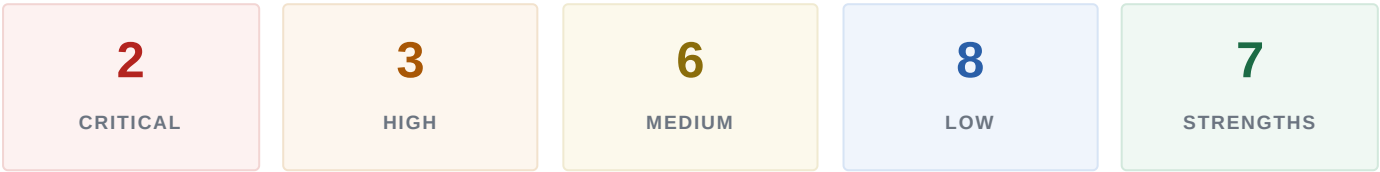


Carselector Test & Review

Full functional, technical and UX assessment of the vehicle-selection funnel at carselector.higaraj.com

APPLICATION	Carselector — <code>carselector.higaraj.com/select-car</code>
SCOPE	Make → Model → Year → Engine funnel and handoff to higaraj.com
METHOD	Live black-box testing: UI walkthrough, API probing, network & console inspection, accessibility audit, responsive checks
DATE	17 September 2026
PREPARED FOR	Motosel



Executive summary

The Carselector app is **well engineered overall** — fast, accessible, secure, and free of console errors. The issues below are specific defects, not a general indictment of build quality. Two of them, however, are commercially serious: one brand is unreachable by browsing, and the "Skip" option silently assigns customers an engine they did not choose.

Testing covered the complete four-step funnel, the search on every step, pagination and infinite scroll, error and empty states, browser navigation and refresh, the API contract and its input validation, security headers, page performance, keyboard and screen-reader accessibility, responsive layout, and the final handoff to the HiGaraj store.

#	FINDING	SEVERITY	AREA
C-1	BMW missing from the brand list (pagination defect)	Critical	Backend / data
C-2	"Skip" still commits the customer to a specific engine spec	Critical	Business logic
H-1	Page refresh destroys all funnel progress	High	State / UX
H-2	No Arabic or RTL support	High	Localisation
H-3	Search fails on the terms customers actually type	High	Search quality
M-1	Intermittent 503 responses on catalog endpoints	Medium	Reliability
M-2	Desktop layout wastes most of the screen	Medium	UX / layout
M-3	No context or confirmation anywhere in the flow	Medium	UX
M-4	Ctrl+A in the search box corrupts the query	Medium	Frontend
M-5	Back arrow on step one is a dead end	Medium	UX
M-6	No client-side caching of catalog responses	Medium	Performance
L-1–8	Data casing, API versioning, dev-host assets, HSTS, redundant search boxes, SEO, and other polish items	Low	Various

Critical findings

C-1 BMW is missing from the brand list — customers cannot find it by browsing

The API reports **142 makes**. Scrolling the complete list returns 142 items, but only **141 are unique**: **Fiat is served twice** and **BMW is never served at all**. BMW is only reachable by typing "bmw" into the search box.

VERIFIED RESULT — PAGING THE FULL LIST AT PAGESIZE=30 (WHAT THE APP USES)

```
apiReportedTotal:      142
itemsReturnedAcrossAllPages: 142
uniqueItems:           141
duplicatedBrands:      ["Fiat"]
bmwInBrowseList:       false
bmwFoundViaSearch:     ["Bmw#301"]  <-- the record exists
```

ROOT CAUSE — VISIBLE AT THE PAGE BOUNDARY

```
page 1 ends:    ... Great Wall|id=334, Fiat|id=324
page 2 starts:  Fiat|id=324, Omoda|id=387, Jaecoo|id=348 ...
```

The ordering is neither alphabetical nor by model count (Icaur, with 2 models, sorts above Volkswagen, with 25), so rows are ordered by a rank or popularity column that contains **ties with no deterministic tiebreaker**. Tied rows reshuffle between the two queries, so one row is served on both pages while another is skipped entirely.

CONFIRMATION — THE LOSS IS PAGE-SIZE DEPENDENT

PAGE SIZE	BRANDS DUPLICATED	BRANDS LOST
30 (live)	Fiat	BMW
50	Jac	Exeed
25	Genesis, Exeed	Genesis, Exeed
12	Tesla, Deepal	Omoda, Jac
7	5 brands	Tesla, Hongqi, Li Auto, Leapmotor, Volvo

RECOMMENDED FIX

Add a deterministic secondary sort key to the makes query — ORDER BY rank DESC, id ASC — or migrate to keyset/cursor pagination. The models endpoint is unaffected; this is specific to /api/catalog/v2/makes .

C-2

"Skip" on the engine step still commits the customer to a specific engine

The engine step offers a **Skip** button under the text "If you're not sure about the engine!". Choosing it still sends a concrete engine spec ID to the store.

HANDOFF URLS CAPTURED FOR THE SAME VEHICLE (TOYOTA, MODEL 4017, 2026)

```
Selected 2.7L:  car_engine=353~2-7-l~inline-4~gasoline    car_spec=71553
Selected 4.0L:  car_engine=306~4-0-l~v6~gasoline        car_spec=96908
Pressed SKIP:   car_engine=any                          car_spec=71553  <- - 2.7L
```

The `car_engine` field correctly signals "unknown", but `car_spec` silently falls back to the first variant in the list. A customer who explicitly told you they don't know their engine is registered as a 2.7L.

Commercial impact: for an oil and servicing business, engine spec determines oil grade and volume. This produces wrong product recommendations for exactly the customers who were honest about being unsure.

RECOMMENDED FIX

Omit `car_spec` entirely (or send an explicit null) when the engine step is skipped, and have the store treat a missing spec as "ask the customer" rather than defaulting silently.

High-priority findings

H-1 Refreshing the page destroys all progress

The URL carries `?step=make` on the first step, then drops its query string entirely for the rest of the funnel. Reloading on the Model, Year or Engine step returns the user to step one with nothing preserved — `sessionStorage` holds only a session UUID, not the selection.

On mobile, where app switching, backgrounding and accidental pull-to-refresh are routine, this silently ends sessions. It also means the funnel cannot be deep-linked, resumed, or shared.

RECOMMENDED FIX

Keep the full selection in the URL (`?step=model&make=418 ...`) and hydrate from it on load, or mirror the selection into `sessionStorage` and restore on mount.

H-2 No Arabic or RTL support

The app is hard-set to `lang="en"` and `dir="ltr"`, with no language switcher and no right-to-left styling. The parent site `higaraj.com` runs `i18next` and initialises with `ar` before switching to `en`, and its header carries an AR/EN toggle.

For an Oman-based business, an English-only funnel sitting in front of a bilingual store is a substantial addressable-market gap. The handoff also drops the locale: it targets `higaraj.com/?...` and is then redirected to `/en/`, so an Arabic visitor would be pushed into English.

RECOMMENDED FIX

Add Arabic translations with proper RTL layout, carry the locale through the funnel, and preserve it in the handoff URL.

H-3

Search returns nothing for the terms customers actually type

Search is a plain substring match against the stored label. Every query below is a realistic customer input, and every one returns an empty "No results for your search" state:

QUERY	RESULT	WHY IT FAILS
mercedes - benz	0 results	The brand's correct spelling; data stores "Mercedes Benz" without the hyphen
vw	0 results	Near-universal abbreviation, no alias
chevy	0 results	The common name for Chevrolet
range rover	0 results	Many owners name the model, not "Land Rover"
landrover	0 results	Missing space breaks the match
lynk & co	0 results	Data stores "Lynk And Co"
cherry	0 results	Common misspelling of Chery, no fuzzy matching
تويوتا (Toyota)	0 results	No Arabic index — compounds H-2
TOYOTA / toyota	Works	Case-insensitivity and trimming are handled correctly

RECOMMENDED FIX

Normalise punctuation and whitespace before matching, add an alias/synonym table (vw → Volkswagen, chevy → Chevrolet, benz → Mercedes-Benz, range rover → Land Rover), index Arabic brand names, and add fuzzy matching for near-misses.

Medium-priority findings

M-1 — Intermittent 503 responses on catalog endpoints

Roughly four 503 responses were observed on `/api/catalog/v2/makes` and `/models` during testing, including one on an initial page load. This was *not* reproducible under deliberate load — 30 concurrent requests all returned 200 — so it appears to be transient upstream instability rather than a capacity limit. When it hits the request the UI is waiting on, the user sees the error screen. Worth correlating against server logs, and worth adding a single automatic retry on the client.

M-2 — Desktop layout wastes most of the screen

At a 1054 px viewport the app renders inside a fixed column roughly 310 px wide, leaving about 370 px of empty space on each side and showing only two models per row. The grid is responsive *within* the column, but the container never widens. Desktop users scroll far more than necessary. Raising the max-width and stepping to 4–6 columns at wider breakpoints would materially shorten the funnel.

M-3 — No context during the flow, no confirmation at the end

Steps 2 to 4 never display what was already chosen; the Model screen simply reads "Select Model" with no mention of Toyota. A user who picked the wrong brand gets no feedback until they reach the end. There is also no summary or confirmation step — completing the funnel drops the user on the generic HiGaraj homepage with no visible indication that their vehicle was registered and no car-specific products surfaced.

M-4 — Ctrl+A in the search box corrupts the query

Pressing Ctrl+A inside the search field does not select the text; it collapses the caret to position 0 (verified: `selectionStart=0, selectionEnd=0`). Subsequent typing is therefore *prepended*. Typing "bmw", then Ctrl+A, then "zzz" yields `zzzbmw` and a no-results screen. Users who reach for select-all to replace a search term get gibberish.

M-5 — Back arrow on step one is a dead end

The in-app back arrow works correctly between steps and preserves selections. On the first step, however, pressing it deselects the chosen brand and then does nothing further. It should either return the user to `higaraj.com` or be hidden on the first step.

M-6 — No client-side caching of catalog responses

Navigating back to the Make step and re-selecting the same brand refetches the identical model list every time — two identical `models?makeId=418` requests were captured for one back-and-forth. Caching responses per make would cut requests and make backtracking feel instant.

Low-priority and polish items

L-1 — Brand names are stored with incorrect casing

The catalog holds `Mg`, `Gmc`, `Byd`, `Zna`, `Mercedes Benz`, `Lynk And Co` and `Im Motors`. The UI masks this with CSS uppercase, but the underlying data is wrong anywhere else it is used — invoices, emails, exports — and it is the direct cause of two search failures in H-3.

L-2 — Inconsistent API versioning

Makes and models live under `/api/catalog/v2/`; years and engines are unversioned at `/api/catalog/`. This will cause confusion during the next migration.

L-3 — Unknown API paths return HTML instead of JSON

`/api/catalog/v2/years` (a plausible but wrong path) returns a full HTML 404 page. API routes should return a JSON error body so clients can parse failures consistently.

L-4 — Production pulls brand logos from a development host

Every make logo is served from `dev.fms.higaraj.com`. A development host sits in the critical render path of a production page, with whatever uptime and caching guarantees that host has.

L-5 — HSTS header missing

`Strict-Transport-Security` is not set. `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy` and `Permissions-Policy` are all correctly present, so this is the one gap in an otherwise solid header set.

L-6 — Redundant search boxes

The Year step offers a search box for 15 items and the Engine step for 2. Both add visual clutter with no benefit; searching a list of two engines is never the faster path.

L-7 — Funnel page is indexable with no canonical

The page is served with `robots: index, follow` and no canonical link. A utility funnel with no standalone content risks being indexed as thin content.

L-8 — Typo on the destination banner (outside Carselector)

Customers see this one second after the handoff: the QPay banner on `higaraj.com` reads **"0% intrestest rate."** — should be "interest". Not a Carselector defect, but it is the first thing a completing customer reads.

What is working well

These are not filler. Several are areas where this app is measurably better than typical production work, and they should be protected during any refactor.

Accessibility is properly implemented

A genuine roving-tabindex radiogroup with `role="radio"` and `aria-checked`, an `aria-live` region announcing "30 options available", an accessible name on every one of 34 buttons, and alt text on every image. This is well above the norm.

Error handling works

Simulating API failure produced a clear "Something went wrong loading vehicles" state with a Retry button that recovered correctly once the endpoint returned. The empty-search state is equally well written.

Input is handled safely

Submitting `<script>alert(1)</script>` through the search field was correctly escaped and executed nothing.

Browser navigation behaves

The browser back button moves correctly between steps and preserves prior selections — which makes the refresh behaviour in H-1 the outlier rather than the pattern.

Mobile fundamentals are right

Tiles are 184×120 px — comfortably above the 44 px touch-target minimum — and the search input is set at 16 px, which prevents the involuntary zoom iOS applies to smaller fields.

API input validation is solid

The endpoints return `400` for oversized, zero, negative, fractional and non-numeric `pageSize` or `page`, and `404` for an out-of-range page. Page size is correctly capped at 50.

Fast, and clean in the console

TTFB 171 ms, DOMContentLoaded 196 ms, load complete 211 ms, with the first page server-rendered. The app emits **zero** console errors or warnings.

Sound implementation details

Search is properly debounced rather than firing per keystroke, infinite scroll loads smoothly, and no broken images were found across all 142 brand logos.

Suggested order of work

PRIORITY	ACTION	EFFORT	EXPECTED BENEFIT
1	C-1 — add a deterministic tiebreaker to the makes sort	Small (backend)	Restores a whole brand to the funnel
2	C-2 — stop sending <code>car_spec</code> when the engine is skipped	Small (backend)	Removes wrong-product risk
3	H-1 — persist funnel state in the URL or session storage	Medium (frontend)	Recovers abandoned mobile sessions
4	H-3 — search aliases, normalisation and fuzzy matching	Medium	Removes common dead ends
5	H-2 — Arabic and RTL support	Large	Opens the primary local market
6	M-2, M-3 — widen desktop layout, add context and a confirmation step	Medium	Shorter funnel, less drop-off
7	M-1, M-4, M-5, M-6 and the L-series polish items	Small each	Robustness and consistency

All findings in this report were reproduced at least twice against the live application. Findings C-1 and C-2 were additionally verified by direct API inspection and by capturing the handoff URLs. Testing was black-box only; no source code was reviewed.